

Refaktoriseringsplan lab 3

Steg 1: Dekomposition av Vehicle(SRP, kodåteranvändning)

- Dekomposition av Vehicle: Göra en Position klass med direction och position metoder samt en Engine klass med engine metoder och attribut samt en Speed klass med speedfactor, incrementspeed/decrementspeed och start/stopEngine.
- Göra så fordon inte kan röra sig om motorn inte är startad.

Steg 2: Bryt ut input hantering(SRP)

- Bryt ut input i en separat klass Inputhandler, som hanterar KeyBoardInput samt eventuell User Input(Spinners, knappar etc) från carView.

Steg 3: Refaktorera DrawPanel så den enbart ritar(SRP)

- Ta bort points från Draw panel, vi har redan positions lagring.
- Ändra moveit så den uppdaterar positionen med bilden utan att veta vilket Vehicle det är. (Minska beroende)
- Gör en klass ImageHandler som hanterar vilken bild som hör till vilket objekt
- Ändra PaintComponent, förenkla så den tar in ett vehicle och repaintar den baserat på dess nuvarande koordinater.

Steg 4: Lägg till interfaces(SRP, SoP, kod återanvändning, beroenden)

- Lägg till turboable, lanemanager, ramp, loadable, loader och cartransporter så CarController kan använda interfaces och inte klasser att referera till.

Steg 5: Refaktorisera CarView(SRP, SoP)

- Göra en break spinner, tydligare.
- Använd metodreferenser, inte ny actionevent för alla metoder/knapptryck.

Steg 6: Refaktorisera CarRepairShop och CarFerry(SRP)

- (Döp om till Ferry och RepairShop och gör så de kan lasta på alla Transportables/loadables och inte bara Car?)
- Göra ett generellt interface LaneManager som kan hantera köer/lanes och återanvändas.

Steg 7: Ändra move-logik i transporters(Kod duplicering)

- Göra en ny metod UpdatePosition/ny Move i Transporter som uppdaterar positionen på allt som är lastat.

Steg 8: Refaktorisera CarController

- Gör "vägg kollisioner" till en egen metod/klass
- Ändra så den använder interfaces och inte klasser i instanceof. (low coupling)
- Även göra en ny simulation klass som lägger till bilarna, repairshop etc och sätter positionerna.
- Logiken blir lättare att återanvända och tydligare.

- **Finns det några delar av planen som går att utföra parallellt, av olika utvecklare som arbetar oberoende av varandra? Om inte, finns det något sätt att omformulera planen så att en sådan arbetsdelning är möjlig?**

Ja, här är ett exempel på hur vår plan skulle kunna utföras:

Utvecklare 1: Dekomposition av Vehicle. (steg 1)

Utvecklare 2: Skapa InputHandler. (Steg 2)

Utvecklare 3: Refaktorisera DrawPanel (Steg 3)

Utvecklare 4: Lägg till interfaces (steg 4)

Utvecklare 5: Refaktorisera CarFerry och CarRepairShop och ändra move logik i transporters (Steg 6 och 7).

Utvecklare 6: Refaktorerar CarView (Steg 5)

Refaktorerar CarController (Steg 8) måste nog göras sist och tillsammans då den är så beroende av allting annat.